# AI Audit Report - HW05 Performance Testing

## 1. AI Tool Information

- AI tool: ChatGPT (GPT-5.6 Luna)
- Scope: performance-testing planning, endurance-test procedure, PowerShell/JMeter commands, demo-video procedure, documentation/report structure, AI analysis planning, and AI critique/audit support
- Important note: this audit records the AI interactions visible in the current conversation context. Where the exact clock time was not preserved, the entry is marked as `exact time unavailable` rather than fabricated.

## 2. Declaration

I used AI tools for the following tasks during HW05:

- planning the HW05 performance-testing work;
- determining the endurance-test procedure;
- preparing PowerShell/JMeter commands;
- preparing the demo-video procedure;
- preparing documentation and report structure;
- discussing AI analysis, human review, and optimization judgments.

## 3. AI Interaction Log

### Interaction 1: Endurance threshold / Task 2 / Task 3 / submission preparation

- Date and time: 2026-08-16 17:23:37 +07:00
- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user asked Codex to:

- determine the endurance threshold by running a short endurance / soak test for around 10 to 15 minutes at sustained load and report concrete numbers such as maximum stable RPS and memory ceiling;
- analyze the results with AI and then critically review the AI output;
- create the main report in Markdown;
- create a README containing the self-assessment table and a test summary report with scenarios run, endpoint groups covered, endurance threshold, bug/performance issue count, and the demo video link.

**AI output summary**

The AI provided a comprehensive Codex prompt instructing Codex to:

- inspect the existing HW05 folder and preserve existing Load, Stress, and Spike work;
- inspect `2026.HW05.Performance Testing_En.md`;
- create or run an approximately 10 to 15 minute endurance JMeter test;

- generate an endurance JMX, raw `.jtl`, HTML report, and summary;
- use actual JTL data as the source of truth;
- monitor CPU, memory, and backend process usage;
- prepare a manual screenshot checklist because Codex cannot capture Windows desktop screenshots;
- document the 3-failed-login lockout/reset procedure;
- analyze actual `.jtl` files with AI;
- create an AI analysis, human misinterpretation review, optimization judgment, and AI critique/audit;
- classify optimization suggestions as feasible, conditionally feasible, or hallucinated/unsupported;
- propose a continuous performance-testing model based on commits, changed files, performance-sensitive changes, JMeter execution, p95 calculation, baseline comparison, and regression detection;
- create a flow chart;
- update `FINAL_REPORT.md` and `README.md`;
- create `FINAL_REQUIREMENTS_AUDIT.md`;
- never fabricate JTL values, screenshots, hardware values, GitHub links, YouTube links, AI conversations, or human review.

## Interaction 2: PowerShell `run-spike.ps1` execution-policy error

- Date and time: 2026-08-16 17:40:29 +07:00
- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user reported that `.\run-spike.ps1` could not be loaded because running scripts is disabled on the system.

**AI output summary**

The AI explained that this was a PowerShell execution-policy problem and recommended:

- `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

followed by:

- `.\run-spike.ps1`

It also suggested:

- `Get-ExecutionPolicy -List`
- `powershell -ExecutionPolicy Bypass -File ".\run-spike.ps1"`

The AI explained that `-Scope Process` only affects the current PowerShell session.

## Interaction 3: Endurance threshold procedure

- Date and time: 2026-08-16 17:53:46 +07:00
- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user asked for a step-by-step procedure to determine the endurance threshold and run a 10 to 15 minute soak test at sustained load.

**AI output summary**

The AI gave a step-by-step procedure:

- use existing Load, Stress, and Spike results to select a reasonable sustained load;
- start the backend;
- set the PowerShell execution policy for the current session;
- create or use an endurance JMeter plan without destroying existing plans;
- configure approximately 10 to 15 minutes of sustained load;
- keep the same end-to-end workflow as the existing HW05 performance tests;
- open Task Manager and monitor CPU, system memory, and backend process memory;
- arrange VS Code and Task Manager in the same frame and capture evidence;
- run the endurance test;
- monitor resource usage during the test;
- allow the test to finish without interference;
- preserve the raw `.jtl` and HTML report;
- analyze throughput, RPS, error rate, average response time, median, p90, p95, p99, and maximum;
- determine whether the sustained load is stable;
- if needed, test progressively higher sustained loads to identify the degradation boundary;
- report the maximum demonstrated stable sustained load rather than claiming an absolute hardware limit.

The AI also emphasized that CPU usage alone is not sufficient to define an endurance threshold and that a test with 100% successful requests may still have poor performance if p95 or latency becomes very high.

## Interaction 4: Existing Load PowerShell script and Endurance script

- Date and time: 2026-08-16 18:13:06 +07:00
- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user showed a Load runner script and asked for an endurance version.

**AI output summary**

The AI proposed an `run-endurance.ps1` based on the existing Load script:

```
param(
  [string]$JMeterHome = $env:JMETER_HOME
)

& (Join-Path $PSScriptRoot "Run-JMeter.ps1") `
    -Plan "23127543_Load_20260813.jmx" `
    -Label "endurance" `
    -Threads "10" `
```

```
    -RampUp "60" `
    -Duration "900" `
    -Loops "-1" `
    -ThinkTimeMs "250" `
    -JMeterHome $JMeterHome
```

The AI explained that 900 seconds equals 15 minutes, that 10 threads means 10 concurrent users for the proposed initial test, and that `-Loops "-1"` was suggested assuming `Run-JMeter.ps1` interprets `-1` as infinite looping.

The AI explicitly recommended inspecting `Run-JMeter.ps1` before relying on the behavior of `-Loops` and `-Duration`.

It also emphasized that a 10-user endurance run alone does not prove that 10 users is the maximum stable threshold.

### Interaction 5: Demo video requirements

- Date and time: 2026-08-16 18:38:13 +07:00
- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user asked for step-by-step instructions to record a demo video: an unlisted YouTube video of at least 6 minutes total, showing the tool and the resource monitor in the same frame, with Vietnamese narration.

**AI output summary**

The AI recommended creating one 7 to 10 minute unlisted YouTube video, or combining clips for Load, Stress, Spike, and Endurance.

It recommended:

- use VS Code as the tool or terminal;
- put VS Code and Windows Task Manager in the same frame;
- show CPU and memory;
- show the backend process (`node.exe`) when possible;
- record Vietnamese narration with the user's own voice;
- use OBS Studio or another screen recorder;
- perform a short microphone/screen test first;
- run each scenario while the tool and resource monitor remain visible;
- narrate the purpose and observed behavior of each scenario;
- document the login lockout/reset if it occurs;
- run the approximately 15-minute endurance test;
- upload the final video to YouTube as Unlisted;
- put the actual YouTube URL in `README.md` and the main report;
- do not fabricate the URL.

It also proposed an approximate timing:

| Section | Suggested duration |
|---|---|
| Introduction | 0:45 |
| Load | 1:15 |
| Stress | 1:30 |
| Spike | 1:30 |
| Endurance | 2:30 |
| Conclusion | 0:45 |
| Total | about 8:15 |

### Interaction 6: Interpretation of 100% successful endurance requests

- Date and time: 2026-08-17 09:23:06 +07:00
- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user asked whether 100% successful requests in endurance means the load is not good and should be increased.

**AI output summary**

The AI explained that 100% successful requests are good, not bad. However, if the goal is to empirically find the endurance threshold, a completely clean test at a low load may only establish a stable point rather than the threshold boundary.

The AI recommended:

- keep the successful test as evidence of stability;
- increase the sustained load progressively;
- use existing Stress results to select a sensible next level;
- check throughput, response time, p95, p99, memory, and resource trends;
- avoid jumping immediately to an extremely high load;
- use the phrase `maximum demonstrated stable sustained load` rather than claiming an absolute hardware limit.

The AI provided an example progression showing how higher loads could be tested to find where degradation begins.

## 4. Earlier HW05-Related AI Interactions in the Available Conversation Context

The following interactions were also visible in the broader HW05 performance-testing conversation context.

### Interaction 7: Performance testing and endurance observations

- Date and time: 2026-08-17 09:37:24 +07:00

- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user asked why memory usage in Task Manager remained high after the load run and when to continue with Spike.

**AI output summary**

The AI explained that high memory immediately after a load test does not automatically mean a memory leak. It can result from:

- Node.js or V8 memory management;
- memory reserved by the process;
- caches;
- garbage collection timing;
- application or database behavior.

The AI recommended observing whether memory continues increasing across repeated runs, remains stable at a high level, or returns toward baseline after sufficient idle time.

It also advised not to start the next scenario merely based on a single high memory reading and to verify that the backend is healthy first.

## Interaction 8: CPU usage during performance testing

- Date and time: 2026-08-17 10:05:57 +07:00
- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user asked whether roughly 10% CPU is okay.

**AI output summary**

The AI explained that approximately 10% CPU is not inherently a problem. Low CPU usage can mean the CPU is not the current bottleneck. Other possible bottlenecks include:

- database operations;
- disk or I/O;
- network;
- application synchronization;
- request concurrency;
- latency;
- locks.

The AI recommended evaluating CPU together with throughput, response time, p95, error rate, memory, and other resources rather than treating CPU percentage as a standalone pass or fail metric.

## Interaction 9: Screenshot evidence for VS Code + Task Manager

- Date and time: 2026-08-17 10:28:09 +07:00
- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user asked whether a screenshot of the tool together with the backend process's resource usage should show VS Code and Task Manager.

**AI output summary**

The AI confirmed that if the JMeter test is being run from a VS Code terminal, a screenshot showing VS Code and Task Manager in the same frame is appropriate evidence.

It recommended showing:

- the JMeter command or test running in VS Code;
- Task Manager at the same time;
- CPU and memory;
- the backend process where possible.

## Interaction 10: Why Node.js may not be obvious in Task Manager

- Date and time: 2026-08-17 10:51:49 +07:00
- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user asked why `node.js` did not appear clearly in Task Manager after running the backend.

**AI output summary**

The AI explained that the Node.js backend process may not be obvious under the simplified Task Manager view. It recommended checking the Details tab and looking for `node.exe`, then identifying the process associated with the backend. It also explained that VS Code itself can have several Node or Electron processes, so the process should be identified carefully rather than assuming any `node.exe` is the backend.

## Interaction 11: PowerShell execution policy

- Date and time: 2026-08-18 13:06:28 +07:00
- AI tool: ChatGPT (GPT-5.6 Luna)

**User prompt**

The user reported that `.\run-load.ps1` could not be loaded because running scripts is disabled on the system.

**AI output summary**

The AI recommended:

- `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

followed by:

- `.\run-load.ps1`

and alternatively:

- `powershell -ExecutionPolicy Bypass -File ".\run-load.ps1"`

## 5. AI-Generated Technical Artifacts / Suggestions Used

The AI provided or suggested the following artifacts during the session:

### Endurance PowerShell script

```
param(
  [string]$JMeterHome = $env:JMETER_HOME
)

& (Join-Path $PSScriptRoot "Run-JMeter.ps1") `
    -Plan "23127543_Load_20260813.jmx" `
    -Label "endurance" `
    -Threads "10" `
    -RampUp "60" `
    -Duration "900" `
    -Loops "-1" `
    -ThinkTimeMs "250" `
    -JMeterHome $JMeterHome
```

Human verification required: the user must verify that `Run-JMeter.ps1` correctly interprets `-Loops "-1"` and `-Duration "900"`.

### PowerShell execution-policy command

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

### Alternative PowerShell command

```
powershell -ExecutionPolicy Bypass -File ".\run-endurance.ps1"
```

### Suggested endurance metrics

- concurrent users or threads;
- throughput or RPS;
- error rate;

- average response time;
- median;
- p90;
- p95;
- p99;
- maximum response time;
- CPU;
- system memory;
- backend process memory;
- resource stability over time.

## 6. AI Limitations / Human Verification

The following points must be verified by the student before submission:

- the endurance threshold must come from actual `.jtl` results, not from AI-generated example values;
- the AI cannot independently prove that a load is the maximum stable load without the actual test data;
- CPU percentage alone is insufficient to determine a performance threshold;
- 100% request success does not automatically mean optimal performance, because p95, p99, and throughput must also be evaluated;
- the proposed `-Loops "-1"` behavior must be checked against the actual `Run-JMeter.ps1`;
- Windows screenshots must be manually captured because the AI did not capture the student's desktop;
- hardware specifications must come from the student's actual machine;
- the GitHub repository URL must be the actual public repository URL;
- the YouTube URL must be the actual unlisted demo URL;
- the human misinterpretation review must be completed by the student after checking AI claims against the raw `.jtl`;
- AI optimization recommendations must be checked against the actual backend architecture before being classified;
- no performance number should be copied from AI examples into the final report.

## 7. Recommended Human Review for the AI Audit

Before submitting this appendix, the student should add:

### Confirmed AI misinterpretations

| # | AI claim | Actual `.jtl` value | Why AI was wrong | Correct interpretation |
|---|---|---|---|---|
| 1 | The AI suggested using `10` threads as the endurance starting point and described it as a possible threshold candidate. | The endurance evidence in the repository is the actual run at `100` threads for `900` seconds with `0` total errors and `21.08` req/s throughput. | The AI was describing a starting hypothesis, not a proven threshold. The actual endurance artifact shows a much higher tested load than the | Treat the AI suggestion as a test-plan starting point only; the measured endurance evidence is the `100`-thread, `900`-second run. |

| # | AI claim | Actual `.jtl` value | Why AI was wrong | Correct interpretation |
|---|---|---|---|---|
| | | | suggested starting point. | |
| 2 | The AI said to use the existing Load/Stress/Spike results to pick a reasonable sustained load and then increase progressively if needed. | The repository now includes an endurance run, but it does not explicitly document the true boundary where performance becomes unacceptable. | The AI gave process guidance rather than a measured threshold. A threshold boundary still cannot be claimed from the current evidence alone. | Keep the guidance as methodology, but do not present it as the measured endurance threshold. |

If no genuine AI misinterpretation is found, state that honestly:

> After checking the AI analysis against the raw `.jtl` files, no material numerical misinterpretation was identified.

### Optimization judgment

| AI recommendation | Evidence in SUT | Classification | Human reasoning |
|---|---|---|---|
| Database index | The backend uses SQLite and the JMeter workflow includes repeated order reads and updates. | Conditional | A database index could help some read-heavy paths, but the repository does not document the exact schema hotspot, so this should be verified against the actual tables and query patterns before claiming it as a fix. |
| Connection pool | The backend evidence in the repository does not document a separate database server or pool configuration. | Hallucinated | With SQLite in the current backend, a traditional connection-pool recommendation is not directly supported by the evidence in the repository. |
| SQLite WAL | The backend uses SQLite and sustains repeated read/write operations during the tests. | Feasible | WAL mode is a plausible SQLite optimization, but it still needs to be validated against the application's transaction pattern and any existing database settings before being accepted as a real recommendation. |

## 8. Final AI Audit Statement

AI was used as an assistance tool during HW05 for performance-testing planning, endurance-test design, PowerShell/JMeter command preparation, interpretation guidance, documentation planning, AI-analysis

planning, and demo-video planning.

The AI-generated suggestions are not treated as authoritative performance measurements. Raw JMeter `.jtl` results and actual test evidence are the source of truth. The student is responsible for verifying numerical results, screenshots, hardware specifications, repository and video URLs, AI interpretations, optimization recommendations, and the final human critique.